INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	**PLATAFORMAS EMERGENTES (e)**				
TÍTULO DE LA PRÁCTICA:	*Aplicaciones Multiplataforma*				
NÚMERO DE PRÁCTICA:	*05*	**AÑO LECTIVO:**	*2026-B*	**NRO. SEMESTRE:**	*X*
FECHA DE PRESENTACIÓN	*25/10/2026*	**HORA DE PRESENTACIÓN**	*11:59 pm*		
INTEGRANTE (s) • Borda Espinoza Gabriela Nayely • Cusilayme Garcia Jose Luis • Mamani Uscamayta, Agustín David				**NOTA (0-20)**	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS

Resumen

Súmate es una aplicación multiplataforma, desarrollada con Flutter y Firebase, que reúne en un solo catálogo las convocatorias de voluntariado y los eventos académicos abiertos de las universidades e instituciones de Arequipa. Nace de un problema concreto: las convocatorias circulan dispersas en cadenas de WhatsApp y afiches, cada una con un formulario distinto, por lo que los estudiantes se enteran tarde o nunca y los organizadores no consiguen suficientes voluntarios ni asistentes. La aplicación permite crear el perfil una sola vez, explorar y filtrar convocatorias, postular con pocos toques y seguir el estado de cada postulación. A los organizadores les permite publicar eventos y ediciones, aceptar o rechazar postulantes y registrar quién participó. A todos les ofrece un historial de participaciones con constancia digital. El mismo código funciona en Android y en navegador web, con tres diseños responsivos. Este informe describe el propósito, los requisitos (completos y pendientes), el reparto de tareas, el proceso de desarrollo por módulo, las interfaces y los anexos. Los requisitos RF01 a RF11 quedaron completos, RF14 parcial, y RF12, RF13, RF15 y RF16 a RF18 pendientes.

Índice

Índice de contenido

1. Descripción general del proyecto

1.1 Propósito del proyecto

1.2 Requisitos funcionales

1.3 Requisitos no funcionales

2. Descripción de las tareas asignadas

3. Descripción del proceso de desarrollo

3.1 Tecnologías, diseño y arquitectura

3.2 Modelo de datos (Firestore)

3.3 Back: servicios, transacciones y reglas de seguridad

3.4 Base visual y diseño responsive (front)

3.5 Módulo 1: Autenticación y perfil (RF01, RF02)

3.6 Módulo 2: Convocatorias (RF03 a RF06, RF14 parcial)

3.7 Módulo 3: Postulaciones (RF07 a RF10)

3.8 Módulo 4: Historial y constancias (RF11)

3.9 Pruebas y verificación

3.10 Alcance recortado, limitaciones y mejoras futuras

4. Interfaces de la aplicación

4.1 Acceso y perfil

4.2 Explorar y detalle de convocatorias

4.3 Interfaces del organizador

4.4 Postulaciones e historial

5. Anexos

Índice de figuras

Figura 1. Prototipo de pantallas de Súmate en Figma (vista general).

Figura 2. Colecciones de Súmate en la consola de Firebase (Cloud Firestore).

Figura 3. Resultado de flutter analyze y flutter test en la terminal.

Figura 4. Inicio de sesión en móvil.

Figura 5. Crear cuenta en móvil.

Figura 6. Perfil del usuario en móvil.

Figura 7. Explorar convocatorias en móvil (390 px).

Figura 8. Explorar convocatorias en tamaño medio (800 px).

Figura 9. Explorar convocatorias en pantalla amplia (1440 px).

Figura 10. Filtros de Explorar en móvil.

Figura 11. Detalle de la convocatoria en móvil.

Figura 12. Detalle de la convocatoria en pantalla amplia.

Figura 13. Formulario Publicar evento.

Figura 14. Mis eventos del organizador.

Figura 15. Detalle de la edición para el organizador.

Figura 16. Gestión de postulantes y asistencia.

Figura 17. Mis postulaciones (pestaña Activas).

Figura 18. Historial de participaciones.

Figura 19. Constancia digital de participación.

Índice de tablas

Tabla 1. Árbol de problemas de Súmate

Tabla 2. Requisitos funcionales y su estado

Tabla 3. Requisitos no funcionales y cómo se atienden

Tabla 4. Tareas por integrante y su estado

Tabla 5. Tecnologías utilizadas

Tabla 6. Colecciones de Firestore

Tabla 7. Breakpoints del diseño responsive

Tabla 8. Pruebas realizadas

Tabla 9. Correspondencia entre requisitos, módulos, pantallas y back

1. Descripción general del proyecto

1.1 Propósito del proyecto

Súmate es una aplicación multiplataforma (Android y web, con un solo código en Flutter) que reúne en un catálogo único las convocatorias de voluntariado y los eventos académicos abiertos de universidades e

instituciones de Arequipa: hackathons, ponencias, lanzamientos, campañas y conferencias. Conecta a tres actores: **organizadores**, **voluntarios** y **asistentes**. La motivación del usuario es el networking, la experiencia, el aprendizaje y la obtención de certificados. Súmate no es una bolsa de trabajo ni de prácticas, y tampoco una red social: no tiene feed, "me gusta", comentarios, seguidores ni chat.

Origen de la idea. Observamos que las convocatorias (por ejemplo, "Convocatoria abierta Hack4Edu UNSA 2026") circulan en cadenas de WhatsApp y afiches, y cada una pide un Google Form distinto con los mismos datos. Antes de decidir evaluamos otras alternativas y las descartamos: repetir la aplicación de matrículas (poca proyección social), un sistema de tutorías (sin incentivo claro) y una bolsa de prácticas (ya existen LinkedIn y Computrabajo). Elegimos un catálogo de convocatorias con postulación y seguimiento porque reutiliza la lógica de los laboratorios anteriores (oferta, cupos, inscripción y estado), lo que da coherencia con el diseño previo del curso.

El problema se organizó en un árbol de problemas (Tabla 1), del cual se derivaron los requisitos, los módulos y las pantallas de la aplicación.

Tabla 1. *Árbol de problemas de Súmate*

Elemento	Descripción
Problema central	Los estudiantes de Arequipa se enteran tarde o nunca de las oportunidades de voluntariado y de los eventos académicos abiertos, y los organizadores tienen dificultad para conseguir voluntarios y asistentes.
Causas	1. Convocatorias dispersas en WhatsApp, afiches y redes sueltas. 2. Cada convocatoria exige un formulario distinto con los mismos datos. 3. No existe un espacio común entre universidades. 4. Los organizadores no conocen las habilidades e intereses de los postulantes. 5. No hay un registro ordenado de participaciones y certificados.
Efectos	1. Menos voluntarios y asistentes de los necesarios; eventos cancelados o vacíos. 2. Estudiantes sin acceso a experiencia, certificados y networking, sobre todo quienes no tienen contactos. 3. Selección de voluntarios lenta y desordenada. 4. Participaciones difíciles de reunir y demostrar (por ejemplo, en LinkedIn).
Enfoque de proyección social	Acceso equitativo a oportunidades de formación y voluntariado, y vínculo entre las universidades y la comunidad.

Aspectos de originalidad. Frente a una simple lista de eventos, Súmate propone cuatro decisiones propias:

- **Modelo Evento y Edición.** "Hack4Edu" es el evento (la serie) y "Hack4Edu 2026" es una edición (su realización). Los eventos únicos son un evento con una sola edición. Cada edición tiene sus propios organizadores, de modo que el líder puede cambiar cada año sin perder el historial.
- **Un solo perfil para todas las convocatorias.** El estudiante completa sus datos una vez y postula con un toque, lo que elimina los formularios repetidos (causa 2 del árbol).
- **Privacidad por copia (snapshot).** Al postular, se guarda una copia del perfil dentro de la postulación; solo los organizadores de esa edición pueden leerla. El perfil completo nunca es público.
- **Historial de participaciones.** Solo cuentan las participaciones que el organizador confirma, y cada una genera una constancia digital (causa 5 del árbol).

- **Un solo tipo de cuenta.** "Organizador" no es un rol: es un permiso sobre una edición. Cualquier usuario puede postular y publicar.

1.2 Requisitos funcionales

Los requisitos se priorizaron como **A** (MVP), **B** (si había tiempo) y **P** (pendiente: solo se documenta). La Tabla 2 indica cuáles están completos y cuáles quedan pendientes. Además de la lista original, se implementaron dos extensiones menores: la recuperación de contraseña (dentro de RF01) y la cancelación de una postulación (dentro de RF07 y RF08).

Tabla 2. *Requisitos funcionales y su estado*

Código	Requisito	Prior.	Estado
RF01	Registro e inicio de sesión (correo y contraseña)	A	Completo
RF02	Perfil con universidad, carrera, ciclo, habilidades e intereses	A	Completo
RF03	Crear evento (serie) y su primera edición con tipo, descripción, fecha, lugar o modalidad, y convocatoria de voluntarios y/o asistentes (cupos, beneficios, fecha límite)	A	Completo
RF04	Enlace externo opcional (por ejemplo, un Google Form)	A	Completo
RF05	Listado de convocatorias con filtros por tipo, universidad, fecha y modalidad	A	Completo
RF06	Detalle de la convocatoria	A	Completo
RF07	Postular o inscribirse con un toque, usando los datos del perfil (incluye cancelar la postulación)	A	Completo
RF08	Ver el estado de mis postulaciones	A	Completo
RF09	El organizador ve a los postulantes con sus habilidades y acepta o rechaza	A	Completo
RF10	El organizador marca quién participó realmente	A	Completo
RF11	Historial de participaciones con constancia digital (en pantalla)	A	Completo
RF12	Guardar convocatorias en favoritos	B	No implementado
RF13	Recordatorios de fecha límite y del evento	B	No implementado
RF14	Editar o cerrar mis ediciones y crear una nueva edición a partir de una anterior	B	Parcial: cerrar y reabrir en la interfaz; editar y "Nueva edición" existen en el back
RF15	Asignar o cambiar los organizadores de una edición	B	No implementado
RF16	Registro de asistencia por QR	P	Pendiente
RF17	Constancia descargable en PDF	P	Pendiente
RF18	Calificar la experiencia del evento	P	Pendiente

En resumen: los 11 requisitos del MVP (RF01 a RF11) están completos; RF14 está parcialmente completo; RF12, RF13 y RF15 no se implementaron por alcance; y RF16 a RF18 quedan como trabajo futuro.

1.3 Requisitos no funcionales

Tabla 3. *Requisitos no funcionales y cómo se atienden*

Código	Requisito	Cómo se atiende en la aplicación
RNF01	Usabilidad	Interfaz minimalista; postular toma 3 toques (tarjeta, "Postular", confirmar). Los estados se muestran con ícono, texto y color, no solo con color.
RNF02	Multiplataforma	Un solo código Flutter para Android y navegador web, con diseño responsive.
RNF03	Rendimiento	Consultas simples por igualdad sobre un solo campo; los filtros se aplican en el cliente, sin índices compuestos. Listas con FutureBuilder y StreamBuilder.
RNF04	Seguridad	Firebase Authentication y reglas de Firestore: cada usuario solo lee y edita lo suyo; solo los organizadores de una edición la modifican y ven a sus postulantes.
RNF05	Disponibilidad	Servicio en la nube de Firebase (plan gratuito Spark), sin servidor propio.
RNF06	Escalabilidad	Las universidades se agregan sin cambiar código (lista con opción "Otra" de texto libre); los tipos de evento se declaran en un único enumerado.
RNF07	Privacidad	El perfil se comparte solo con los organizadores de las ediciones a las que se postula, mediante una copia (snapshot) dentro de la postulación; la colección usuarios es privada.
RNF08	Mantenibilidad	Código por módulos (features/), tema y widgets centrales en core/ y una sola clase Servicios con una instancia de cada servicio.

2. Descripción de las tareas asignadas

El trabajo se repartió en tres áreas: **idea, diseño y documentación** (Gabriela Borda Espinoza), **front en Flutter** (José Luis Cusilayme García) y **back en Firebase** (Agustín David Mamani Uscamayta). La Tabla 4 detalla las tareas de cada integrante e indica si quedaron completadas o pendientes.

Tabla 4. *Tareas por integrante y su estado*

Integrante	Tarea	Req. / módulo	Estado
Gabriela Borda Espinoza (idea, diseño y documentación)	Definir el problema, el árbol de problemas, el propósito y el enfoque de proyección social	—	Completada
	Redactar y priorizar los requisitos funcionales (A, B, P) y no funcionales	RF01–RF18, RNF01–RNF08	Completada
	Diseñar el prototipo (12 pantallas móvil y 12 web) y el sistema de diseño "Warm Civic"	Todos los módulos	Completada
	Definir el diseño responsive (breakpoints y navegación principal)	RNF01, RNF02	Completada
	Elaborar la especificación del proyecto y el presente informe	—	Completada
José Luis Cusilayme García (front en Flutter)	Tema central Material 3, utilidad responsive, widgets reutilizables y AppShell	RNF01, RNF02, RNF08	Completada
	Módulo 1: AuthGate, inicio de sesión, registro, recuperación de contraseña y perfil	RF01, RF02	Completada
	Módulo 2: Explorar con filtros, detalle de convocatoria, publicar evento y Mis eventos	RF03–RF06	Completada

	Postular y cancelar postulación desde el detalle	RF07	Completada
	Módulo 3: Mis postulaciones y gestión de postulantes y asistencia	RF08–RF10	Completada
	Módulo 4: historial y constancia digital en pantalla	RF11	Completada
	Cerrar y reabrir la convocatoria desde el detalle del organizador	RF14	Completada
	Editar la edición y "Nueva edición" desde la interfaz	RF14	Pendiente
	Favoritos, recordatorios y asignación de organizadores	RF12, RF13, RF15	Pendiente
	Pruebas automáticas del front y verificación en Chrome, emulador y teléfono Android	—	Completada
Agustín David Mamani Uscamayta (back en Firebase)	Configurar Firebase (Authentication y Firestore) y el emulador local	RNF04, RNF05	Completada
	Diseñar el modelo de datos, los modelos Dart y los enumerados	Todos los módulos	Completada
	AuthService y UsuarioService	RF01, RF02	Completada
	EventoService y EdicionService (creación atómica de evento y primera edición)	RF03–RF06, RF14	Completada
	PostulacionService: transacciones de cupos, plazo y duplicados; respuesta del organizador; asistencia; cancelación	RF07–RF11	Completada
	Reglas de seguridad de Firestore y su despliegue	RNF04, RNF07	Completada
	Actualización en tiempo real de postulaciones (streams de Firestore)	RF08, RF09	Completada
	Datos de demostración (8 eventos) y regresión del back en el emulador (29 casos)	—	Completada
	Asistencia por QR, constancia en PDF y calificaciones	RF16–RF18	Pendiente

3. Descripción del proceso de desarrollo

El desarrollo siguió el orden **diseño, back y front por módulos**. Primero se definieron el problema, los requisitos y el prototipo; luego se construyó la capa de datos (modelos, servicios y reglas) y se probó en el emulador de Firebase; por último se construyó la interfaz módulo por módulo, verificando cada uno antes de pasar al siguiente. Los fragmentos de código de este capítulo son ilustrativos y están simplificados.

3.1 Tecnologías, diseño y arquitectura

Tabla 5. *Tecnologías utilizadas*

Componente	Tecnología	Uso en Súmate
Framework	Flutter 3.47.6 (Dart), Material 3	Interfaz única para Android y web (flutter create --platforms=android,web)
Autenticación	Firebase Authentication	Registro e inicio de sesión con correo y contraseña

Base de datos	Cloud Firestore	Documentos, transacciones, batches, streams y reglas de seguridad
Paquetes	firebase_core, firebase_auth, cloud_firestore, google_fonts	Conexión a Firebase y tipografía Plus Jakarta Sans
Estado y navegación	FutureBuilder, StreamBuilder, setState, Navigator	Sin paquetes adicionales de rutas ni de estado
Diseño	Google Stitch y Figma	Prototipo de pantallas y sistema de diseño
Pruebas	Emulador de Firebase, flutter test, flutter analyze	Regresión del back y pruebas del front

Diseño. El prototipo (12 pantallas móvil y 12 web) se elaboró antes de programar y está disponible en Figma (ver Anexos). El sistema de diseño "Warm Civic" toma una paleta cálida inspirada en el sur peruano: Verde Sabio #ABC270, Mostaza #FEC868, Coral #FDA769, Café Expreso #473C33 y fondo Crema #FBF8F1. La tipografía es Plus Jakarta Sans y se evita el negro puro. Los textos sobre los colores de marca usan siempre el Café Expreso para mantener el contraste.

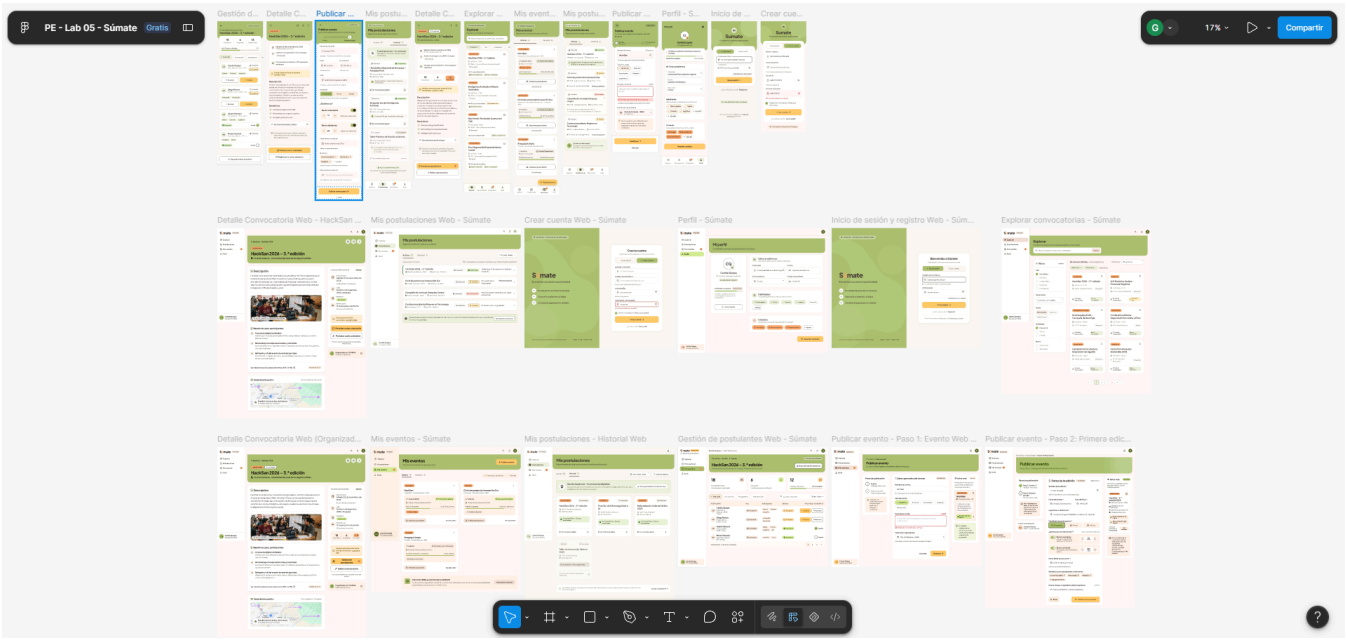


Figura 1. *Prototipo de pantallas de Súmate en Figma (vista general).*

Decisiones de arquitectura.

- **Sin servidor propio.** El back usa solo Firebase (plan Spark, sin Cloud Functions). Las reglas de negocio (cupos, plazos, duplicados) se resuelven con transacciones de Firestore y las de acceso con reglas de seguridad.
- **Navegación simple.** Se usa Navigator con un AuthGate reactivo a la sesión, sin go_router: no agrega paquetes y es fácil de explicar. El costo es que en la web no hay URLs compartibles por pantalla.
- **Estado sin librerías.** FutureBuilder y StreamBuilder con setState; una clase Servicios expone una instancia única de cada servicio.

- **Interfaz delgada.** La interfaz solo habilita o deshabilita acciones y muestra los mensajes (en español); las reglas de negocio residen en el back.

La estructura de carpetas separa lo común (core) de los módulos (features):

```
Shell
  lib/
    main.dart                        // inicializa Firebase
    app/                             // auth_gate.dart, app_shell.dart
    core/
        theme/                       // app_colors.dart, app_theme.dart
        responsive.dart, servicios.dart, formato.dart
        widgets/                     // evento_card, estado_badge, filtro_chip, ...
    features/
        auth/ perfil/ eventos/ postulaciones/   // modelos, servicios, pantallas
  test/                              // modelos, formato, layouts, cancelación
```

3.2 Modelo de datos (Firestore)

El modelo tiene cuatro colecciones (Tabla 6). La convocatoria no es una colección: son los objetos voluntarios y asistentes dentro de cada edición, con los campos abierta, cupos, ocupados, beneficios, fechaLimite y enlaceExterno.

Tabla 6. *Colecciones de Firestore*

Colección	ID del documento	Campos principales	Propósito
usuarios	UID de Authentication	nombre, correo, universidad, carrera, ciclo, habilidades[], intereses[]	Perfil privado; uno solo para todas las postulaciones
eventos	Generado	nombre, tipo, descripcion, entidad, administradores[]	La marca o serie (“Hack4Edu”)
ediciones	Generado	eventoId, nombre, fechaInicio, fechaFin, lugar, modalidad, universidad, estado, organizadores[], voluntarios{}, asistentes{}	Una realización concreta (“Hack4Edu 2026”); guarda copias de eventoNombre y tipo
postulaciones	{edicionId}_{usuarioId}_{rol}	estado, asistio, fecha, perfil{} (snapshot), eventoNombre, edicionNombre, fechaInicio, lugar	Postulación; el historial son las postulaciones con asistio == true

- **Id determinista** de la postulación: impide que un usuario postule dos veces con el mismo rol a la misma edición.
- **Denormalización:** el nombre y la fecha del evento se copian a la edición y a la postulación para listar sin consultas adicionales.
- **Snapshot del perfil** dentro de la postulación: el organizador ve las habilidades del postulante sin acceder a su perfil completo (RNF07).

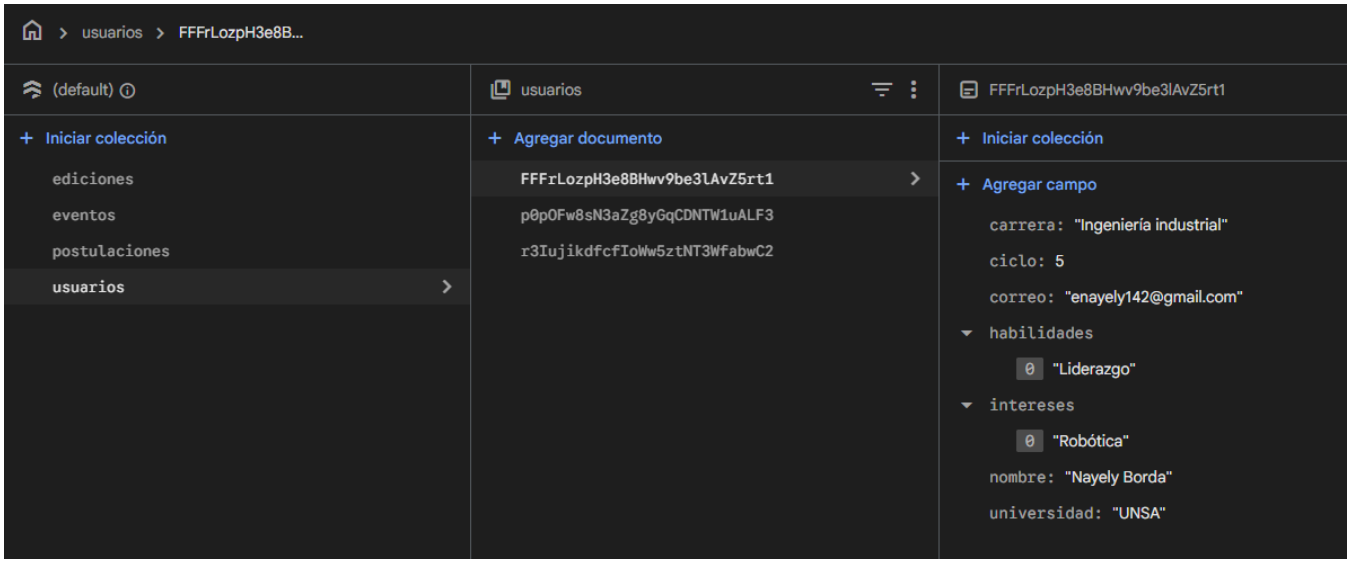


Figura 2. *Colecciones de Súmate en la consola de Firebase (Cloud Firestore).*

3.3 Back: servicios, transacciones y reglas de seguridad

La capa de datos se construyó antes que la interfaz y se probó en el emulador de Firebase. Cada servicio (AuthService, UsuarioService, EventoService, EdicionService, PostulacionService) encapsula las consultas de su módulo y traduce los errores a mensajes en español (AuthError y ErrorNegocio).

- **Transacciones (runTransaction)** en postular, responder, marcarAsistio y cancelar: validan cupo, plazo y duplicados y actualizan el contador de cupos de forma atómica. Los asistentes ingresan como "aceptada" y descuentan cupo al postular; los voluntarios quedan "enviada" y el cupo se descuenta cuando el organizador acepta.
- **Batch (WriteBatch)** en crearEventoConPrimeraEdicion: crea el evento y su primera edición en una sola operación, sin estados intermedios.
- **Reglas de seguridad (firestore.rules)**: un usuario solo lee y edita su perfil; cualquier usuario autenticado lee eventos y ediciones; solo los organizadores modifican una edición y leen las postulaciones de esa edición.
- **Consultas sin índices compuestos:** se filtra por un solo campo y el resto de filtros se aplica en el cliente.

Dart (unsupported)

```
// PostulacionService.postular (simplificado)
await _db.runTransaction((tx) async {
  final edicion = Edicion.fromFirestore(await tx.get(edicionRef));
  if (!edicion.admitePostulaciones(rol)) throw ErrorNegocio('Convocatoria cerrada');
  if ((await tx.get(postRef)).exists) throw ErrorNegocio('Ya postulaste');
  tx.set(postRef, postulacion.toMap());     // incluye el snapshot del perfil
  if (rol == RolPostulacion.asistente) {
      tx.update(edicionRef, {'asistentes.ocupados': FieldValue.increment(1)});
  }
});
// firestore.rules (simplificado): lee el dueño o un organizador de la edición
match /postulaciones/{pid} {
  allow read: if request.auth != null &&
      (resource.data.usuarioId == request.auth.uid ||
      request.auth.uid in edicionDe(resource.data.edicionId).organizadores);
}
```

3.4 Base visual y diseño responsive (front)

Antes de las pantallas se construyó la base común, que da consistencia (RNF02) y facilita el mantenimiento (RNF08).

- **Tema central:** un único ThemeData Material 3 con el ColorScheme armado a mano (no con fromSeed), para respetar exactamente la paleta de marca, y la tipografía con google_fonts. Los colores viven solo en AppColors.
- **LayoutBuilder + AppShell:** elige el tipo de navegación según el ancho disponible. Los cuatro destinos (Explorar, Postulaciones, Mis eventos y Perfil) se definen una sola vez y los usan ambos diseños; IndexedStack conserva el estado de cada pestaña.
- **Widgets reutilizables:** EventoCard, EstadoBadge, FiltroChip, PostulacionCard, PostulanteCard, CabeceraPantalla y WordmarkSumate, basados en Card, InkWell y FilterChip.

Tabla 7. *Breakpoints del diseño responsive*

Rango de ancho	Navegación	Columnas de la lista	Margen

Compacto (< 600 px)	NavigationBar inferior (4 destinos)	1	20 px
Medio (600–1023 px)	NavigationRail compacto (solo íconos)	2	24 px
Amplio (≥ 1024 px)	Riel lateral extendido de 240 px	3	32 px

Dart (unsupported)

```
// AppShell (simplificado)
LayoutBuilder(builder: (context, c) {
  if (c.maxWidth < Breakpoints.medio) {         // < 600 px
      return Scaffold(body: pagina, bottomNavigationBar: NavigationBar(...));
  }
  return Scaffold(body: Row(children: [
      c.maxWidth >= Breakpoints.amplio ? RielAmplio(...) : NavigationRail(...),
      Expanded(child: pagina),
  ]));
})
```

Durante las pruebas se corrigió un error de diseño: el tema definía el botón con Size.fromHeight(48), que da un ancho mínimo infinito y dejaba en blanco la pantalla Explorar al usarlo dentro de un Row. Se cambió a Size(0, 48).

3.5 Módulo 1: Autenticación y perfil (RF01, RF02)

- **Qué se hizo:** inicio de sesión, registro, recuperación de contraseña y perfil editable con cierre de sesión.
- **AuthGate con StreamBuilder** sobre los cambios de sesión: mantiene la sesión al recargar y evita consultar Firestore sin usuario autenticado.
- **Form y TextFormField** con validaciones; los errores de Firebase llegan ya traducidos al español. La recuperación de contraseña usa un AlertDialog.
- **Perfil:** DropdownButtonFormField para la universidad (UNSA, UCSM, UCSP, "Otra" con texto libre e "Independiente"), chips editables para habilidades e intereses, y SnackBar para confirmar los cambios.
- **Registro seguro:** crea el usuario en Authentication y su documento en usuarios; si la escritura falla, elimina el usuario para no dejar cuentas sin perfil.

Dart (unsupported)

```
StreamBuilder<User?>(
  stream: Servicios.auth.cambiosSesion,
  builder: (context, snap) {
```

```
        if (snap.connectionState == ConnectionState.waiting) {
        return const CircularProgressIndicator();
        }
        return snap.data == null ? const PantallaLogin() : const AppShell();
    },
)
```

3.6 Módulo 2: Convocatorias (RF03 a RF06, RF14 parcial)

- **Explorar (RF05):** FutureBuilder con RefreshIndicator y una GridView de 1, 2 o 3 columnas según el ancho. Los filtros por tipo, modalidad, universidad y fecha "desde" usan FilterChip y showDatePicker; en móvil se agrupan en un showModalBottomSheet. Se filtra en el cliente para evitar índices compuestos y se excluyen las ediciones ya finalizadas.
- **Detalle (RF06) y postulación (RF07):** muestra datos de la edición y del evento, cupos y fecha límite. El botón "Postular" se habilita con Edicion.admitePostulaciones(rol) y pide confirmación con un AlertDialog. El enlace externo (RF04) se muestra con SelectableText y el botón "Copiar enlace" (Clipboard). Si el perfil está incompleto, se muestra un aviso que no bloquea la postulación, para mantener los 3 toques.
- **Publicar evento (RF03, RF04):** un solo formulario con dos secciones (Evento y Primera edición), con showDatePicker, showTimePicker y SwitchListTile para activar voluntarios y/o asistentes. Sugiere el nombre de la edición ("evento + año") y la hora de fin (inicio + 4 horas). El back crea ambos documentos en un batch.
- **Mis eventos y detalle del organizador (RF14 parcial):** pestañas "Activas" y "Pasadas" con TabBar, insignia "N por revisar" y FloatingActionButton.extended para publicar. El detalle del organizador muestra métricas calculadas en el cliente (postulantes, en espera, aceptados, asistieron y cupos) y permite cerrar y reabrir la convocatoria con confirmación; en pantallas anchas usa una distribución 8/12 + 4/12.

3.7 Módulo 3: Postulaciones (RF07 a RF10)

- **Mis postulaciones (RF08):** lista con StreamBuilder sobre un stream de Firestore, de modo que las postulaciones nuevas y los cambios de estado aparecen al instante. Inicialmente se usaban Future, pero IndexedStack conserva las pestañas y los datos no se actualizaban hasta cerrar sesión.
- **Estados accesibles:** EstadoBadge combina ícono, texto y color (en espera, aceptada y rechazada).
- **Cancelar postulación:** desde el detalle de la convocatoria. Un voluntario solo puede cancelar mientras está "en espera"; un asistente, hasta la fecha de inicio. Al cancelar un asistente, el cupo se libera en la misma transacción.
- **Gestión de postulantes (RF09, RF10):** PostulanteCard con el snapshot del perfil (universidad, carrera, ciclo y habilidades), filtros por estado y rol, botones Aceptar/Rechazar (solo para voluntarios, ya que los asistentes nacen aceptados) y la marca "Asistió", que solo se habilita desde la fecha de inicio.

3.8 Módulo 4: Historial y constancias (RF11)

- **Una sola pantalla con pestañas** (DefaultTabController, TabBar y TabBarView): "Activas" e "Historial".
- **Historial:** son las postulaciones con asistio == true; la tarjeta muestra "Asististe" en lugar del estado.
- **Constancia digital:** un showDialog compone la constancia con los datos de la postulación (perfil, evento, edición, fecha y lugar). La versión descargable en PDF (RF17) queda pendiente.

3.9 Pruebas y verificación

Tabla 8. *Pruebas realizadas*

Prueba	Alcance	Resultado
Regresión del back en el emulador de Firebase	Autenticación, servicios, transacciones y reglas de seguridad	29 de 29 casos aprobados
Cancelación de postulaciones	6 casos de reglas en el emulador y 4 pruebas de interfaz	Aprobados
flutter test	Modelos, formato de fechas y cupos, layouts responsive a 390, 800 y 1440 px, cancelación	37 pruebas aprobadas
flutter analyze y flutter build web	Análisis estático y compilación web	Sin observaciones; compilación correcta
Pruebas manuales	Chrome (390, 800 y 1440 px), emulador Android y teléfono Android real; reglas desplegadas en la base real	Flujo completo funcional

El flujo de demostración completo es: publicar evento, explorar, postular, aceptar o rechazar, marcar asistencia, ver el historial con constancia y cancelar una postulación. No se probó en iPhone: el proyecto se creó solo para Android y web.

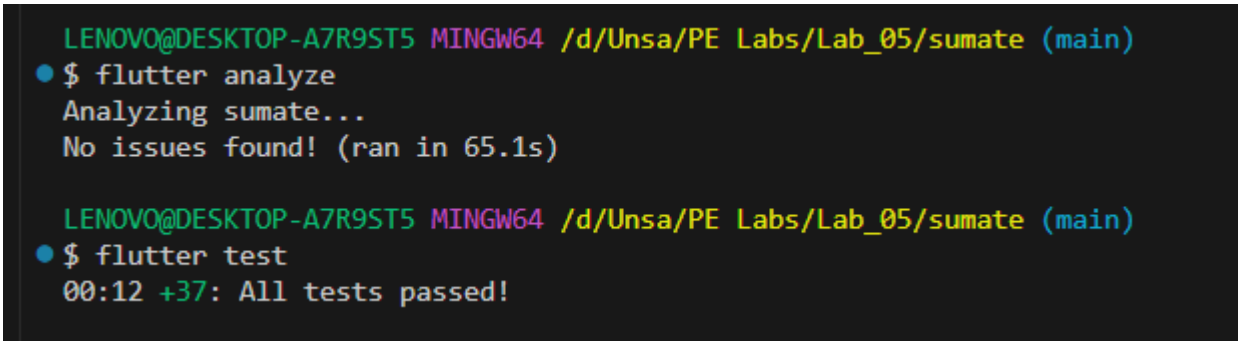


Figura 3. *Resultado de flutter analyze y flutter test en la terminal.*

3.10 Alcance recortado, limitaciones y mejoras futuras

- No se implementaron favoritos (RF12), recordatorios (RF13) ni la asignación de organizadores (RF15).
- RF14 es parcial: falta editar la edición y "Nueva edición" en la interfaz (ya existen en el back).
- No se registran horas de voluntariado: el back guarda solo si la persona asistió (asistio).
- El enlace externo no se abre directamente, solo se copia (no se incluyó url_launcher).
- En la web no hay URLs compartibles por pantalla, por la navegación simple con Navigator.

- Un voluntario aceptado no puede cancelar por su cuenta: debe contactar al organizador.
- Los datos de demostración vencen el 25/11/2026; después se deben crear ediciones con “Publicar evento”.
- Trabajo futuro: asistencia por QR (RF16), constancia en PDF (RF17) y calificación del evento (RF18).

4. Interfaces de la aplicación

La Tabla 9 muestra la correspondencia entre requisitos, pantallas y servicios del back, y confirma que cada función tiene su interfaz y que cada interfaz responde a un requisito derivado del árbol de problemas. Las pantallas tienen cuatro destinos principales (Explorar, Postulaciones, Mis eventos y Perfil), iguales en móvil y web.

Tabla 9. *Correspondencia entre requisitos, módulos, pantallas y back*

Req.	Módulo	Pantalla	Servicio del back
RF01	1. Autenticación y perfil	pantalla_login, pantalla_registro	AuthService
RF02	1. Autenticación y perfil	pantalla_perfil	UsuarioService
RF03, RF04	2. Convocatorias	pantalla_publicar_evento	EdicionService.crearEventoConPrimeraEdicion
RF05	2. Convocatorias	pantalla_explorar	EdicionService.listarAbiertas
RF06	2. Convocatorias	pantalla_detalle_convocatoria	EdicionService.obtener, EventoService.obtener
RF07	3. Postulaciones	pantalla_detalle_convocatoria	PostulacionService.postular, cancelar
RF08	3. Postulaciones	pantalla_mis_postulaciones (Activas)	PostulacionService.escucharMisPostulaciones
RF09	3. Postulaciones	pantalla_gestion_postulantes	postulantesDeEdicion, responder
RF10	3. Postulaciones	pantalla_gestion_postulantes	PostulacionService.marcarAsistio
RF11	4. Historial y constancias	pantalla_mis_postulaciones (Historial), dialogo_constancia	PostulacionService.escucharHistorial
RF03, RF14	2. Convocatorias	pantalla_mis_eventos, pantalla_detalle_organizador	EdicionService.misEdiciones, cerrar, reabrir

4.1 Acceso y perfil

Las pantallas de acceso (RF01) piden solo los datos necesarios y muestran los errores en español. El perfil (RF02) se completa una vez y se reutiliza en todas las postulaciones.



Figura 4. *Inicio de sesión en móvil.*



Figura 5. *Crear cuenta en móvil.*



Figura 6. *Perfil del usuario en móvil.*

4.2 Explorar y detalle de convocatorias

Explorar (RF05) es la pantalla principal. Se adapta al ancho: lista de una columna en móvil, dos columnas en tamaño medio y tres columnas con riel lateral en pantallas amplias.



Figura 7. *Explorar convocatorias en móvil (390 px).*

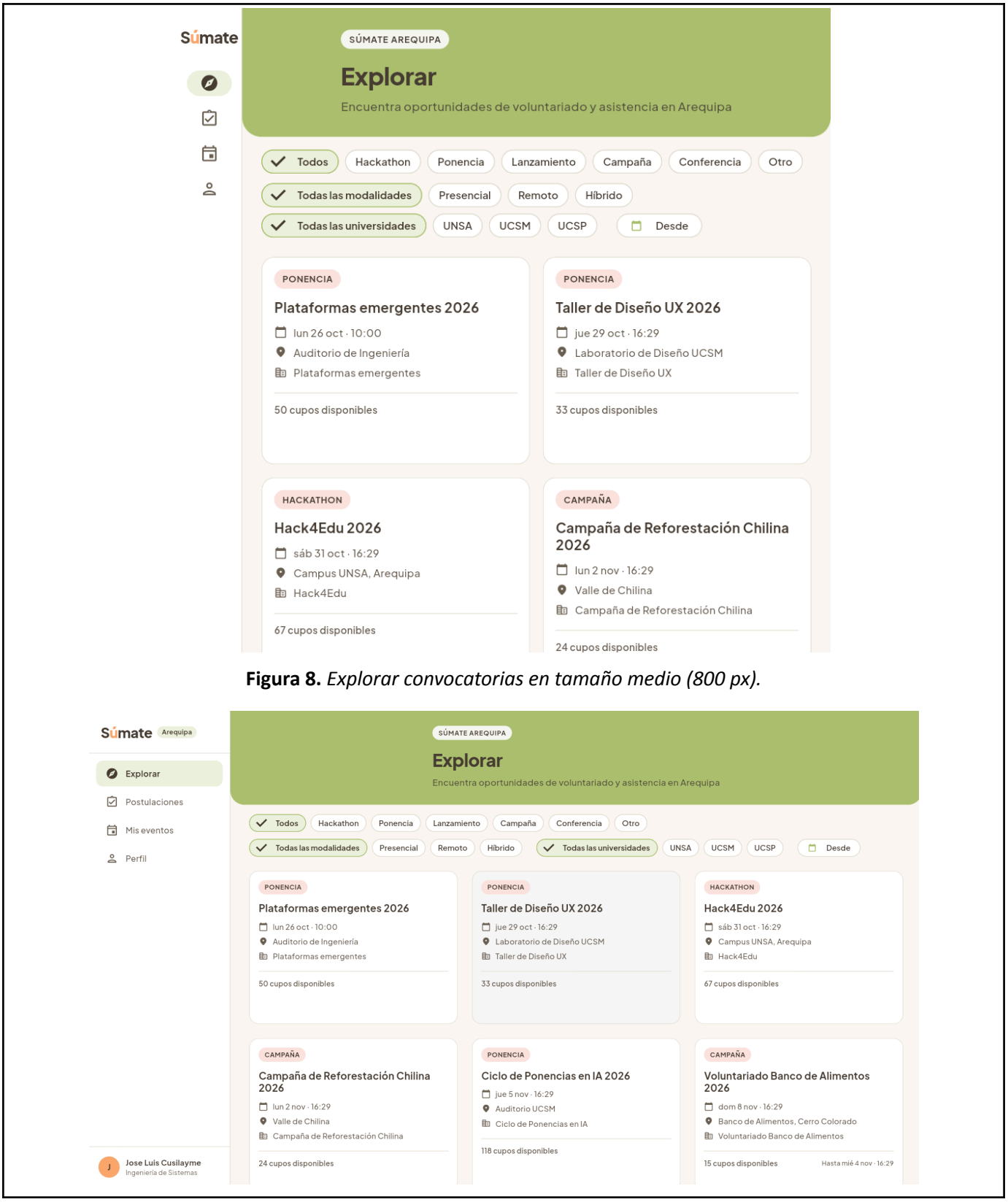


Figura 8. *Explorar convocatorias en tamaño medio (800 px).*

Figura 9. *Explorar convocatorias en pantalla amplia (1440 px).*



Figura 10. *Filtros de Explorar en móvil.*

El detalle (RF06) concentra la información para decidir y la acción de postular (RF07) en pocos toques.



Figura 11. *Detalle de la convocatoria en móvil.*

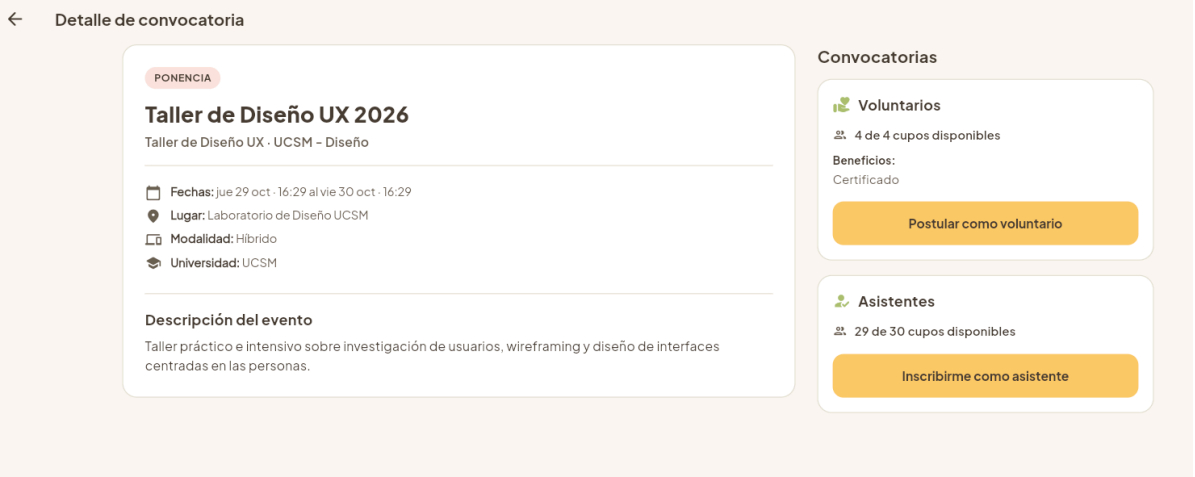


Figura 12. *Detalle de la convocatoria en pantalla amplia.*

4.3 Interfaces del organizador

El organizador publica eventos (RF03, RF04), revisa sus ediciones (RF14) y gestiona a los postulantes (RF09, RF10).

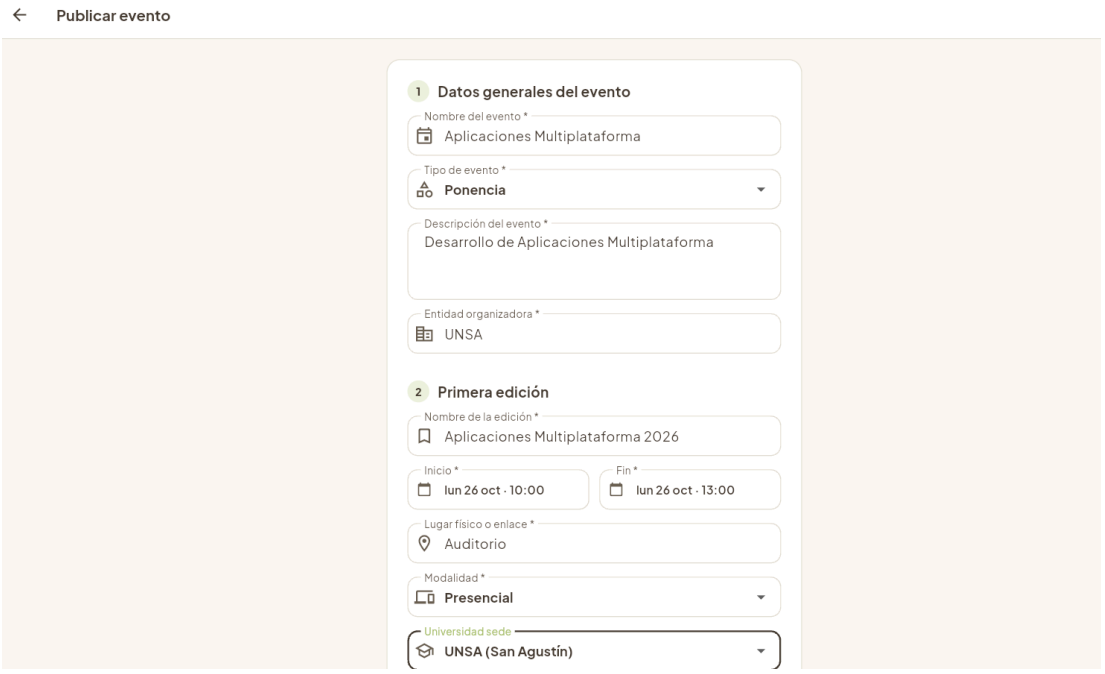


Figura 13. *Formulario Publicar evento.*

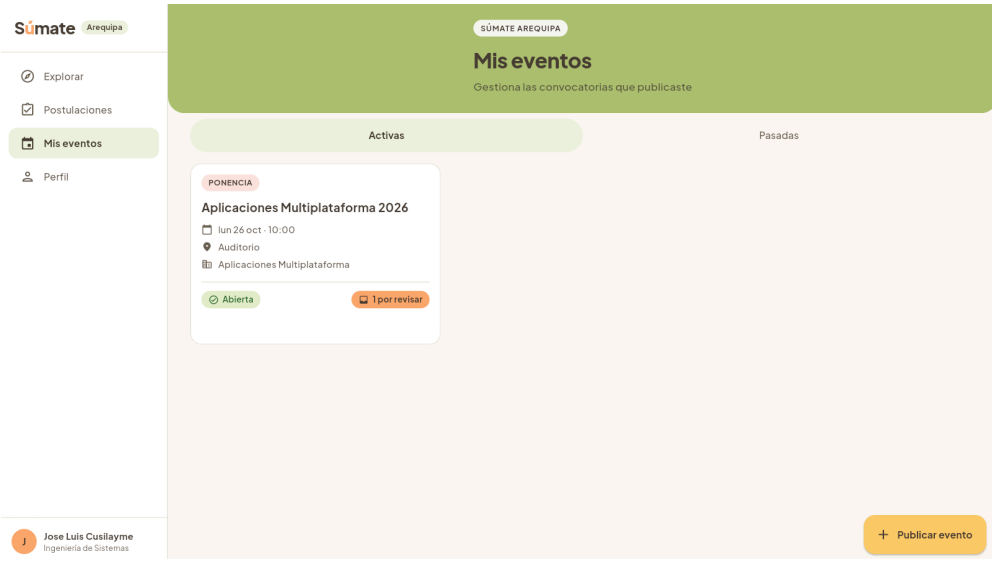


Figura 14. *Mis eventos del organizador.*

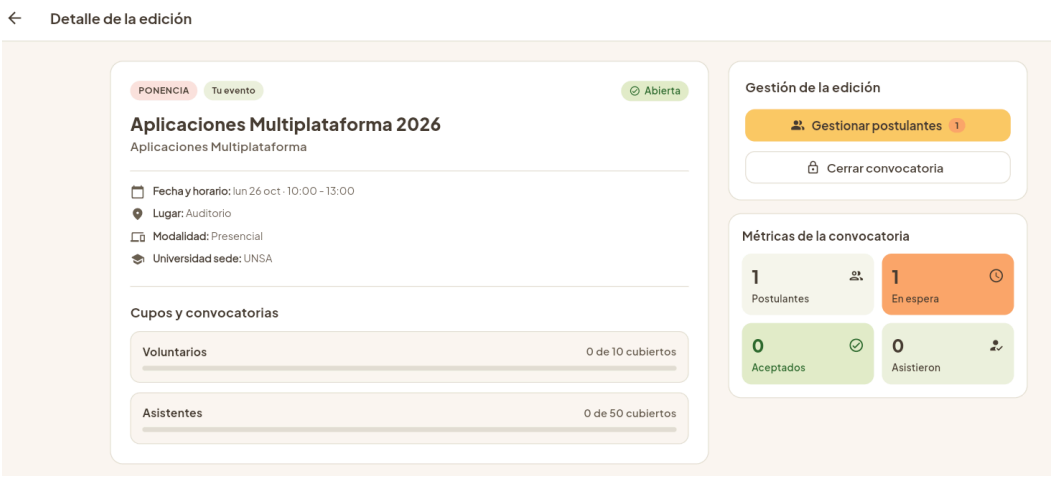


Figura 15. *Detalle de la edición para el organizador.*



Figura 16. *Gestión de postulantes y asistencia.*

4.4 Postulaciones e historial

El estudiante sigue el estado de sus postulaciones (RF08) y conserva su historial con constancia (RF11).



Figura 17. *Mis postulaciones (pestaña Activas).*



Figura 18. *Historial de participaciones.*



Figura 19. *Constancia digital de participación.*

5. Anexos

- **Repositorio en GitHub:**
  https://github.com/gbordae/plataformas-emergentes-labs/tree/main/Lab_05/sumate (proyecto en la carpeta Lab_05/sumate, con README).
- **Prototipo de diseño en Figma:**
  https://www.figma.com/design/q7vDIvP6u8kdPkaHTFC7NX/PE---Lab-05---S%C3%BAmate?node-id=0-1&t=j1qgF7UqWUhHtdzf-1
- **Video de demostración o ejecutable (APK):**
  https://drive.google.com/drive/folders/1jSjw8rWM1lt0BcAJJwBryjoamru_KmuK?usp=sharing
- **Cuenta de demostración:** demo@sumate.pe / 123456 (organizadora de las 8 ediciones de ejemplo: Hack4Edu, Ciclo de Ponencias en IA, Lanzamiento Startup Arequipa, Campaña de Reforestación Chilina, Conferencia de Ciberseguridad, Feria de Proyectos Tecnológicos, Taller de Diseño UX y Voluntariado Banco de Alimentos). Para probar la asistencia hay que crear una edición con inicio de hoy.

Cómo ejecutar la aplicación:

```
Shell
flutter pub get

flutter run -d chrome              # web

flutter devices && flutter run -d <id>   # Android
```

I. CUESTIONARIO:

No aplica

CONCLUSIONES

Súmate cumple el objetivo del laboratorio: es una aplicación multiplataforma, hecha con Flutter y Firebase, que resuelve un problema real con enfoque de proyección social.

- **Resultado.** Los 11 requisitos del MVP (RF01 a RF11) están completos y el flujo de demostración funciona de principio a fin: publicar, explorar, postular, aceptar, marcar asistencia, ver el historial con constancia y cancelar. RF14 quedó parcial y RF12, RF13, RF15 y RF16 a RF18 quedan como trabajo futuro.
- **Multiplataforma.** Un solo código sirve a Android y a la web, con tres diseños responsivos (barra inferior, riel compacto y riel extendido), un tema Material 3 central y widgets reutilizables.
- **Decisiones de modelo.** Separar el evento de sus ediciones, usar un perfil único con copia al postular y guardar el historial como postulaciones confirmadas resolvió tres causas del árbol de problemas: formularios repetidos, falta de privacidad y ausencia de un registro de participaciones.
- **Aprendizaje técnico.** Mover las reglas de negocio al back (transacciones y reglas de seguridad) mantuvo delgada la interfaz. También se comprobó que IndexedStack exige datos en tiempo real (StreamBuilder) y que un detalle del tema, como Size.fromHeight, puede romper un layout completo.
- **Mejoras futuras.** Completar la edición de ediciones, favoritos, recordatorios, asignación de organizadores, asistencia por QR, constancia en PDF, calificación del evento y registro de horas de voluntariado.

METODOLOGÍA DE TRABAJO

- **Análisis:** se definió el problema y su árbol, y se priorizaron los requisitos (A, B y P).
- **Diseño:** se elaboró el prototipo de 12 pantallas móvil y 12 web y el sistema de diseño “Warm Civic”.

- **Construcción:** primero el back (modelos, servicios, reglas y pruebas en el emulador de Firebase) y luego el front módulo por módulo, verificando cada módulo con flutter analyze y flutter test antes de seguir.
- **Herramientas de apoyo:** se trabajó con desarrollo asistido por IA (Claude para planificar y revisar; Antigravity y Cursor para escribir código) a partir de un documento de especificación; el equipo revisó cada entrega en el código y con las pruebas.
- **Validación y entrega:** pruebas manuales en Chrome, emulador y teléfono Android, y entrega en GitHub.

REFERENCIAS Y BIBLIOGRAFÍA

Firebase. (s. f.). Cloud Firestore. Google. https://firebase.google.com/docs/firestore
Firebase. (s. f.). Firebase Authentication. Google. https://firebase.google.com/docs/auth
Firebase. (s. f.). Firebase Local Emulator Suite. Google. https://firebase.google.com/docs/emulator-suite
Firebase. (s. f.). Cloud Firestore Security Rules. Google. https://firebase.google.com/docs/firestore/security/get-started
Flutter. (s. f.). Flutter documentation. Google. https://docs.flutter.dev
Flutter. (s. f.). Adaptive and responsive design in Flutter. Google. https://docs.flutter.dev/ui/adaptive-responsive
Flutter. (s. f.). NavigationBar class y NavigationRail class [API]. Google. https://api.flutter.dev/flutter/material/
Google. (s. f.). Material Design 3. https://m3.material.io
Google Fonts. (s. f.). Plus Jakarta Sans. Google. https://fonts.google.com/specimen/Plus+Jakarta+Sans
Pub.dev. (s. f.). google_fonts [paquete de Dart]. https://pub.dev/packages/google_fonts